Evaluating Large Language Models for Automated Code Refactoring

Andres Ricardo Ardila Agudelo
Concordia University
Montreal, QC, Canada
a_ardil@live.concordia.ca

Diego Palacios Escalera
Concordia University
Montreal, QC, Canada
di_pala@live.concordia.ca

Ahkil Dhim
Concordia University
Montreal, QC, Canada
ak_dhim@live.concordia.ca

Abstract - Code refactoring is a critical yet time-consuming software engineering task aimed at improving readability, maintainability, and performance without altering a program's functionality. This study evaluates the effectiveness of three code-focused Large Language Models (LLMs)—CodeLlama-7B, StarCoder2-7B, and WizardCoder-7B—for automated code refactoring. Using a dataset of 420 Python programs from Aizu Online Judge, we compare models under zero-shot, one-shot, and few-shot prompting. We assess correctness via a custom virtual judge (Pass@k) and code quality using metrics such as LOC, CC, Halstead Complexity, Maintainability Index, and Levenshtein Distance. CodeLlama, especially with few-shot prompts, consistently outperforms others in both correctness (Pass@5 = 0.70) and code maintainability. Prompting strategy significantly affects outcomes, with few-shot prompting yielding cleaner and more concise refactorings. A developer survey confirms alignment between human preferences and models that improve readability and structure. While LLMs show strong promise, post-processing and developer oversight remain essential for safe integration into real-world workflows.

I. Introduction

As software systems evolve, the importance of code refactoring also increases. This practice plays a crucial role in improving code maintenance by analyzing and reworking existing code to enhance readability, maintainability, and overall performance; all without changing its functionality. [1] A good practice of this process leads to an easier debugging phase and adaptation to the change of the requirements that a software project might face during its development. However, the refactoring task has been for the most part a manual process that requires deep expertise and detailed analysis to avoid introducing new errors or compromising the original code in some type of way that ruins its functions, and thus, being more time consuming.

At the same time, the rise of artificial intelligence, particularly through Large Language Models (LLMs), introduced transformative possibilities in automating and improving different fields of software engineering. These technologies have demonstrated an impeccable potential for understanding natural language, code maintenance and even code generation. [2] Well known LLMs, like OpenAI's ChatGPT models, have shown great capabilities in understanding and generating code across different programming languages. Their ability to recognize patterns, interpret context, and suggest optimizations positions them as promising tools for code refactoring tasks in the long run. Research has explored the use of LLMs for code completion, bug and error detection, and even complex program synthesis. [3] However, when it comes to code refactoring, their application remains vaguely explored, as previous studies suggest that achieving precise efficiency results often depends on specific coding environments inside development projects instead of a general evaluation based on ordinary datasets. [4]

Recent studies have begun to explore this frontier. With well-crafted prompts and contextually rich examples, LLMs can achieve high accuracy in transforming code. For example, experiments using GPT-3.5 with few-shot prompting achieved refactoring success rates as high as 95.68%, with significant improvements in code quality metrics [5][6]. Building on this line of work, our study investigates the use of more specialized code-oriented LLMs to evaluate whether such gains hold across broader tasks and models.

Code refactoring is an essential practice in software engineering aiming at the enhancement of code readability, maintainability, and overall performance without the necessity of changing its core functionality. Despite all the benefits this practice offers to the development and maintenance of a software project, manual refactoring is often considered a highly risky and expensive practice as a constant substantial effort is required to ensure behavior preservation [7]. Similar approaches to this topic have stated that the manual application of code refactoring tends to be very time and resource consuming as well as prone to human mistakes in an introduction to the code environment, leading to the underutilization of this practice in large-scale projects [8].

Therefore, there is a need to develop methodologies that can boost reliability and developer trust in LLM code refactoring. Such methodologies should not just aim for automation accuracy but also for developer oversight, that way ensuring that the refactoring suggestions are well aligned with project-specific requirements and different coding standards. Addressing these challenges could potentially lead to more efficient development processes, fewer manual entry problems, less manual effort in general, and improved project quality.

II. Related Work

In recent years, the application of *Large Language Models* (LLMs) to software engineering tasks has sparked considerable interest, especially in areas such as code completion, bug detection, and automated code synthesis. Within this growing body of research, automated code refactoring—where the structure of code is improved without altering its functionality—has emerged as a promising yet underexplored frontier. Unlike traditional rule-based tools that follow fixed transformation rules, LLMs offer a flexible, learning-based approach that can adapt to diverse coding patterns and styles. This flexibility, however, comes with concerns regarding consistency, reliability, and semantic correctness, particularly when the models are used in high-stakes development environments.

Shirafuji et al. (2023) represent one of the earlier empirical investigations into the feasibility of using LLMs for code refactoring. Their work focuses on GPT-3.5 and evaluates the effectiveness of few-shot prompting to guide the model in transforming Python programs. The study demonstrates that even without fine-tuning, LLMs can generate valid and meaningful refactorings, achieving notable improvements such as a **25.84%** reduction in average lines of code and a **17.35%** decrease in cyclomatic complexity. These metrics suggest that LLMs are capable of producing more concise and potentially more maintainable code, especially when supplied with well-designed examples. However, the authors also document critical limitations: the model sometimes introduces redundant transformations, overly simplified logic, or generates comments that do not reflect the developer's original intent. Furthermore, while the performance was promising on small-scale examples, the study indicates a drop in quality and reliability when LLMs are applied to more complex or real-world software systems. This highlights the lack of control and verifiability in LLM-generated refactoring—a key gap our study addresses by integrating both automated metrics and human evaluations [6].

In a more comprehensive follow-up to such studies, *Liu et al.* (2024) provide an extensive empirical analysis of LLMs' capabilities in automated software refactoring. Their methodology includes not only performance comparisons between LLMs and traditional refactoring tools but also a multidimensional evaluation strategy incorporating maintainability metrics, structural code changes, and developer feedback. One of the major strengths of this study is the integration of a human-in-the-loop evaluation framework, where developers assess the readability, clarity, and maintainability of LLM-generated outputs. Their results confirm that LLMs are particularly effective at stylistic and syntactic improvements—such as variable renaming, redundant code elimination, and formatting—but struggle with deeper structural transformations that require understanding program context or architectural intent. Additionally, the study finds that the lack of transparency in how LLMs make transformation decisions can hinder developer trust, especially in large-scale or safety-critical software systems. These limitations underline the importance of interpretability and control mechanisms, which are often missing in current LLM applications [5].

Building on these foundational works, this research adopts a more systematic and comparative approach to evaluating LLMs in automated refactoring. Going beyond prior work by benchmarking three specialized code-focused models—*CodeLlama, StarCoder2,* and *WizardCoder*—across a standardized dataset of 420 functionally correct Python programs. Unlike *Shirafuji et al*., who focused on a single model and prompt format, this study evaluates multiple prompting strategies (*zero-shot, one-shot,* and *few-shot*) to assess how prompt engineering influences refactoring effectiveness. Moreover, the analysis is not limited to traditional metrics like *lines of code* and *cyclomatic complexity,* including *Halstead complexity*, *maintainability index*, and *Levenshtein distance*, complemented by human reviews to assess output quality from a developer's perspective.

By incorporating a virtual judge system for correctness validation and leveraging a diverse metric suite for quality assessment, we address a major limitation identified in both previous studies: the absence of standardized, reproducible evaluation protocols. Our pipeline is designed not only to evaluate model performance but also to expose trade-offs between aggressiveness and safety in refactoring. This helps reveal whether certain LLMs tend to produce minimal yet safe transformations, or if others pursue more ambitious rewrites that risk compromising functionality or maintainability. In doing so, we contribute novel insights into the real-world applicability of LLMs for refactoring and offer practical guidance for integrating them into modern software engineering workflows.

III. Research Questions

To systematically evaluate the potential and limitations of LLMs in automated code refactoring, this study focuses on the following research questions:

1. **How do different LLMs compare in refactoring success rate and code reduction?**
2. **How do LLMs balance correctness, readability, and maintainability in automated code refactoring?**
3. **How does prompt engineering affect refactoring performance?**

Answering these research questions will give us a clearer picture into the capabilities, trade-offs, and practical applicability of LLMs in automated code refactoring, which is an essential aspect of software engineering. By comparing different LLMs in terms of success rate and code reduction (**RQ1**), we will identify which models are most effective at simplifying code while maintaining correctness. Some models might be great at simplifying code but introduce subtle bugs, while others might be more cautious, making smaller, safer improvements. Knowing these differences will help developers choose the best LLM for integrating automated refactoring into their workflows.

Beyond just reducing lines of code, readability and maintainability (**RQ2**) are critical. Clean, efficient code is useless if no one can understand or modify it later. Poorly structured refactoring, even if functionally correct, can negatively impact software maintainability and developer productivity. By analyzing how LLMs handle these trade-offs, we'll see whether they truly make code better in the long run.

Finally, by exploring the impact of prompt engineering (**RQ3**), we will determine how different prompting strategies

influence refactoring quality, showing whether fine-tuned prompt designs can significantly improve results or if model limitations restrict effectiveness. Understanding these aspects will help developers optimize their interactions with LLMs, making AI-assisted software development more reliable and practical.

These findings will contribute to a better understanding of how LLMs can support software engineering tasks, what challenges remain, and what improvements are needed for them to become trustworthy, efficient, and scalable tools in professional software development environments. These findings will contribute to a better understanding of how LLMs can support software engineering tasks, what challenges remain, and what improvements are needed for them to become trustworthy, efficient, and scalable tools in professional software development environments.

IV. Methodology

To evaluate the effectiveness of different *Large Language Models* (LLMs) in code refactoring, our methodology is structured around three key research questions. For **RQ1**, which compares LLMs in refactoring success rate and code reduction, we begin by applying a standardized refactoring prompt to a curated dataset of 420 functionally correct Python programs sourced from the *Aizu Online Judge* (AOJ) [10]. We evaluate the performance of three state-of-the-art LLMs: *CodeLlama* [12], *StarCoder2* [13], and *WizardCoder* [14]. Each model generates multiple refactored variants per input program.

We will assess refactoring success through *Pass@k* [15], a metric that evaluates whether at least one of the generated k candidates is functionally correct. Correctness will be determined through a virtual judge system, which runs hidden test cases to verify that refactored outputs maintain the original program's behavior. To measure code reduction, we will track *Lines of Code* (LOC) reduction, *Cyclomatic Complexity* (CC) [16], and token reduction, all of which serve as indicators of code conciseness and complexity minimization. This analysis will reveal which models produce the most effective code transformations while maintaining correctness, helping assess their suitability for real-world integration in automated refactoring workflows.

For **RQ2**, which investigates how LLMs balance correctness, code reduction, readability, and maintainability, we will extend the analysis of functionally correct refactored outputs from **RQ1**. In addition to complexity and code reduction metrics, we will incorporate *Halstead Complexity* [17] and *Maintainability Index* [18] to measure maintainability and *Levenshtein Distance* [19] to evaluate the degree of transformation applied by each LLM. A higher distance may indicate aggressive modifications that could compromise maintainability, while a lower distance suggests more conservative refactoring. [19] To complement these automated metrics, we also conduct a human evaluation, where experienced developers review and rank selected refactored outputs based on readability, maintainability, and overall code quality. This step provides insights into whether LLM-generated refactorings align with human expectations for clean, sustainable code.

For **RQ3**, which examines the impact of prompt engineering on refactoring effectiveness, we will apply four prompting strategies: *zero-shot*, *one-shot* and *few-shot* [20]. These strategies will be tested on a subset of programs to measure their effect on refactoring success. Metrics such as *Pass@k, LOC,* and *CC* reduction, will be used to compare the effectiveness of different prompts, to determine whether more complex prompts lead to better refactoring and whether the improvements justify potential increases in computational cost. By comparing how different LLMs respond to different prompting methods, we aim to identify the most effective approach for guiding automated code refactoring.

V. Dataset

To evaluate the effectiveness of multiple *Large Language Models* (LLMs) in automated code refactoring, we construct a dataset consisting of functionally correct programs sourced from *Aizu Online Judge* (AOJ). Our dataset selection process follows a structured methodology, including data collection, preprocessing, and filtering to ensure quality and diversity in refactoring scenarios.

The *AOJ* platform provides a repository of over 8 million code submissions, including both accepted and rejected solutions, primarily from programming courses and competitive programming contests. For our purposes, we focus exclusively on the *Introduction to Programming I* (ITP1) course, which includes a range of problem types from basic syntax exercises to algorithmic challenges.

The source code submissions to *AOJ* are publicly available for research and educational purposes. These can be accessed either directly from the official *AOJ* source archive or through aggregated datasets such as *CodeNet* [22], which indexes submissions from multiple platforms including *AOJ*.

A. Data Collection

We collect correct Python submissions that are marked as Accepted by AOJ's online judge system, thereby ensuring functional correctness. Our initial filtering process yielded a total of **94,150** accepted Python programs corresponding to ITP1 problems.

However, certain problems (e.g., *ITP1_1_A Hello World* and *ITP1_3_A Print Many Hello World*) were not found in the source archive and were therefore excluded from the dataset. These exclusions had minimal impact on overall dataset coverage.

B. Preprocessing

After collecting the correct programs, we apply preprocessing steps to refine the dataset for experimentation. This includes duplicate removal and random selection, ensuring a representative and manageable dataset.

a) Duplicate Removal

To avoid redundant data, we eliminate duplicate programs within each problem set. This is performed by comparing raw source code at the character level. Programs that differ only in whitespace or formatting are treated as duplicates and removed. After this deduplication step, the dataset was reduced to **68,531** unique programs, forming the final base corpus for our refactoring experiments.

b) Random Selection

Finally, to ensure diversity while maintaining a manageable dataset size, we perform a uniform random sampling of 10 unique programs per problem from the deduplicated pool. Although the Introduction to Programming I course originally contained 44 programming problems, two problems (*ITP1_1_A* and *ITP1_3_A*) were excluded due to missing source files. This results in a total of 42 problems used for our study. By selecting 10 programs per remaining problem, our final dataset comprises 420 Python programs.

C. Final Dataset

The final dataset consists of **420 functionally correct Python programs**, each representing a unique solution to one of the 42 selected ITP1 problems. The problems span topics such as control structures, loops, arrays, and basic algorithms, providing a varied and representative foundation for evaluating LLM-based code refactoring. Each program will serve as input for evaluating multiple LLMs on refactoring success rate, code reduction, readability, and maintainability.

VI. Experimental Setup

Our experimental pipeline is designed to automate the evaluation of LLM-generated refactorings across a consistent set of inputs, models, and prompting strategies. The key components include the virtual judge system, model configuration, prompt engineering, and *Pass@k* output generation.

A. Virtual Judge System

To validate functional correctness of refactored code, we implement an offline virtual judge system. Test cases for each problem are retrieved via the *Aizu Online Judge* (AOJ) API and stored in structured JSON format—one file per problem. We develop a local Python-based judge that executes both original and refactored programs against these hidden test cases. Only programs that pass all test cases are considered correct. All 420 programs in the final dataset were validated using this judge prior to LLM evaluation.

B. Model Configuration

We focus on three state-of-the-art code-oriented Large Language Models:

- CodeLlama-7B[12]
- StarCoder2-7B[13]
- WizardCoder-7B[14]

Due to hardware limitations and the desire for local inference, we use the 7B parameter versions of each model. All models were downloaded in March 2025, and were run locally on a machine equipped with an Intel Core i9 CPU and an NVIDIA RTX 4050 GPU, using the *Ollama framework* [24], which allows easy model interaction via Python.

Each model is queried via a custom Python script that:

- Loads the original .py source file
- Constructs a prompt tailored to the selected prompting strategy
- Calls the LLM using ollama.chat
- Saves the refactored outputs into organized output directories by model, problem ID, and pass number (e.g., pass@1, pass@3, pass@5)

C. Prompt Engineering

We evaluate the impact of prompt design using three prompting strategies:

- **Zero-shot**: Only a direct refactoring instruction is given.[25]
- **One-shot**: Includes one worked example of a refactoring transformation before the target program.[26]
- **Few-shot**: Includes three worked examples to guide the LLM toward more effective refactorings.[27]

Each prompt ends with a strict instruction for the LLM to return only Python code (no explanations, no formatting, no markdown).

Example – Zero-Shot Prompt (excerpt):

Below is a Python program that works as intended, but I need you to refactor the code to reduce its lines and complexity, while preserving its functionality. ...
Here is the original Python program:
{python_code}
Please provide the refactored version of the code ...
DO NOT include any explanations ... Just give me the Python code, no quotations.

One-shot and *few-shot* prompts prepend one or more refactoring examples following the same format, illustrating desirable code transformations.

D. Pass@k Output Generation

To compute *Pass@k*, we generate five distinct refactorings per program using repeated queries to the LLM. These are stored as:

- *pass@1.py* – the first refactored output
- *pass@3.py, pass@5.py* – subsequent outputs for evaluation

Each output is independently evaluated using the virtual judge. *Pass@k* is computed by checking whether at least one of the top k generated outputs is functionally correct.

E. Evaluation Metrics

To assess the impact of LLM-based refactoring beyond functional correctness, we compute several static code metrics using a custom Python script built on the Radon library:

- **Lines of Code (LOC)**: Counts the number of executable lines in the program, used to measure code size reduction.[16]
- **Cyclomatic Complexity (CC)**: Quantifies the number of independent execution paths in the code, reflecting its logical complexity.[17]
- **Halstead Complexity**: Measures cognitive effort based on the number and variety of operators and

operands, providing insight into code understandability and maintainability.[18]

- **Maintainability Index (MI)**: A composite metric (based on LOC, CC, and Halstead values) scaled to indicate how easy the code is to maintain—higher values suggest better maintainability.[19]
- **Levenshtein Distance**: Computes the minimum number of single-character edits needed to transform the original code into the refactored version, serving as a proxy for the degree of syntactic change.[28]

These metrics enable a multi-faceted analysis of the refactoring impact in terms of brevity, simplicity, and long-term maintainability.

F. Pipeline Summary

The complete evaluation pipeline is structured to automate the end-to-end process of assessing LLM performance in code refactoring. It includes the following key stages:

1) Dataset Preparation:

Functionally correct Python programs are collected from AOJ, preprocessed to remove duplicates, and randomly sampled to create a balanced dataset of 420 unique programs.

2) Prompted Model Inference:

Each program is submitted to three LLMs—*CodeLlama, StarCoder2,* and *WizardCoder*—using multiple prompting strategies (*zero-shot, one-shot, few-shot*). For each input, the models generate up to five refactored versions.

3) Refactored Output Generation:

Outputs are stored in a structured format (*pass@1, pass@3, pass@5*) per model and prompt type, allowing for reproducible experiments and scalable evaluation.

4) Virtual Judge Evaluation:

Each refactored output is validated against hidden test cases using a custom offline judge. This determines *Pass@k*, indicating whether at least one of the generated variants preserves the original behavior.

5) Code Quality Analysis:

Functionally correct outputs are further analyzed using static analysis metrics—*Lines of Code (LOC), Cyclomatic Complexity (CC), Halstead Complexity, Maintainability Index (MI),* and *Levenshtein Distance*—computed via the *Radon* library.

G. Implementation Reproducibility

To ensure reproducibility, all scripts used are modularized and version-controlled. The codebase is organized to support easy extension for new models or metrics. All scripts and experimental artifacts will be made publicly available via *GitHub* to encourage transparency and facilitate future research.[23]

H. Developer Evaluation of Refactored Code

To complement the quantitative evaluation of model outputs, we conducted a user study to understand how developers perceive refactored code in practice. First, a random sample of **40** code pairs (original and refactored) was selected. Each participant was shown **10** randomly selected pairs in a side-by-side, unlabeled format with randomized ordering (A/B). Participants were not informed which version was original or refactored.

For each pair, participants selected their preferred version—Image A, Image B, or indicated no preference. In addition to their selection, they were asked to specify one or more reasons for their choice, choosing from: more readable, more concise, or easier to maintain.

VII. Results

A. RQ 1 - Comparing LLM Performance

1) Correctness

We compared three language models—*CodeLlama, WizardCoder,* and *StarCoder2*—on a benchmark of automated code-refactoring tasks, evaluating their performance using the *Pass@k* metric. This measures the proportion of problems correctly solved when considering up to k generated outputs per task. The evaluation focused on three budgets: *k = 1, 3,* and *5*. [Table 1]

Among the models, *CodeLlama* demonstrated the strongest performance across all values of k and prompting strategies. Under *few-shot* prompting, it achieved a *Pass@1* of **43%**, which increased to **58%** at *Pass@3*, and reached **70%** at *Pass@5*. *WizardCoder* followed with more modest scores, achieving a maximum *Pass@5* of **24%** under *zero-shot* prompting. *StarCoder2*, by contrast, exhibited consistently poor results, never exceeding **4%** at *Pass@5* under any condition.

These results highlight a clear distinction in refactoring reliability: *CodeLlama* not only produces significantly more correct outputs on the first attempt, but it also benefits the most from increased generation budget. The 27-point gain from *Pass@1* to *Pass@5* demonstrates *CodeLlama's* ability to recover from initial inaccuracies. In contrast, *WizardCoder* shows more limited improvements across multiple attempts, while *StarCoder 2* shows little to no progression, suggesting a fundamental limitation in its ability to perform this task effectively.

Overall, the results strongly suggest that *CodeLlama* is the most capable and adaptable model for automated code refactoring, particularly in contexts where correctness is critical and multiple response candidates can be leveraged to improve success rates.

Model	Prompting	Pass@1	Pass@3	Pass@5
Codellama	zero	0.28	0.41	0.51
	one	0.33	0.54	0.64
	few	0.43	0.58	**0.7**
Starcoder2	zero	0	0.01	0.02
	one	0.02	**0.04**	**0.04**
	few	0	0.01	0.01
Wizardcoder	zero	0.11	0.17	**0.24**
	one	0.08	0.19	0.23
	few	0.13	0.19	0.21

Table 1: Pass@k Accuracy for Each Model and Prompting Strategy

Model	Prompting	LOC original	LOC refactored	CC original	CC refactored	Halstead original	Halstead refactored	MI original	MI refactored	Levenshtein Distance
Codellama	zero	14.67	10.6	0.38	0.48	64.75	57.68	71.42	73.28	103.19
	one	13.02	11.16	0.34	0.4	56.92	53.65	70.47	71.79	44.34
	few	13.95	11.18	0.34	0.32	58	52.4	70.26	71.19	64.82
Starcoder2	zero	11.4	11.4	0	0	71.45	70.96	74.18	82.67	58.7
	one	13.5	11	0	0	48	48	88.64	88.15	4.5
	few	4	3	0	0	4.75	4.75	84.72	84.72	2
Wizardcoder	zero	10.02	7.92	0.19	0.46	37.35	31.7	75.58	79.23	84.7
	one	8.11	6.78	0.17	0.36	27.87	28.81	75.71	77.6	46.39
	few	8.29	6.06	0	0	40.66	34.71	73.64	78.15	45.71

Table 2: Code Quality Metrics for Accepted Refactorings by Model and Prompting Strategy (Average)

2) Lines of Code Reduction

We also evaluated the conciseness of accepted refactored outputs by comparing their *Lines of Code* (LOC) against the original implementations. This analysis was restricted to

successfully refactored programs—i.e., those that passed the automated evaluation.

Across all models and prompting configurations, a consistent reduction in LOC was observed, indicating that all three models tend to produce more compact code upon successful refactoring. The most substantial reductions were achieved by *CodeLlama* and *WizardCoder*, both of which showed a clear drop in LOC across all settings. Few-shot prompting appeared to further encourage concise outputs, particularly in the case of *CodeLlama*, where the gap between original and refactored LOC remained steady regardless of the input length.

StarCoder 2 showed similar trends, but its overall impact was limited due to the small number of successful refactorings. In summary, the models not only varied in correctness but also in their ability to simplify code, with CodeLlama again demonstrating the most favorable balance of correctness and conciseness. [Table 2]

3) Cyclomatic Complexity Analysis

We analyzed cyclomatic complexity to evaluate the logical structure and control flow of refactored code, focusing

only on successful outputs. This metric helps indicate whether the refactoring led to simplification or introduced additional branching and conditional logic.

For both *CodeLlama* and *WizardCoder*, most refactored outputs exhibited higher complexity compared to the original code. This increase suggests that while the refactoring may have improved correctness or reduced lines of code, it often introduced additional control paths. Notably, the only setting where complexity decreased was *CodeLlama* (*few-shot*), where the average complexity dropped slightly compared to the original. This reduction indicates cleaner, more structured logic in those cases—likely benefiting from the additional contextual cues provided during prompting.

StarCoder 2 showed zero complexity across all runs, both before and after refactoring. This is consistent with earlier findings where the model only succeeded on trivial programs, implying that the tasks it managed to refactor were likely simple, linear code without branching.

Overall, while increases in complexity were common, the exception seen with *CodeLlama* in the *few-shot* setting is promising, highlighting its capacity to produce not just correct but also cleaner and more maintainable code under the right prompting conditions. [Table 2]

B. RQ2 - Readability and Maintainability

1) Halstead Volume Analysis

To evaluate maintainability from a complexity standpoint, we analyzed the *Halstead Volume* of accepted refactored code. This metric quantifies the size of the implementation in terms of operators and operands, with higher values typically reflecting more intricate or harder-to-understand code.

Across all models and prompt types, a general decrease in *Halstead Volume* was observed in the refactored versions. This trend suggests that, when successful, the refactorings led to simplified logic and reduced operational complexity. *CodeLlama* demonstrated consistent reductions across all prompting conditions, with the most significant drop seen in the few-shot setting. *WizardCoder* also showed notable reductions in volume, particularly under the zero-shot and few-shot settings.

StarCoder 2, despite its limited correctness in earlier evaluations, produced refactored code with comparable or slightly lower *Halstead Volume* in the few cases where it succeeded. These findings support the view that both *CodeLlama* and *WizardCoder* can generate not only correct but also more maintainable code, at least from a Halstead complexity standpoint. [Table 2]

2) Maintainability Index

To further assess maintainability, we computed the *Maintainability Index* (MI) for all accepted refactored outputs. This composite metric captures structural clarity by combining *cyclomatic complexity*, *lines of code*, and *Halstead complexity* into a single interpretable value. Higher scores indicate better maintainability.

Overall, we observed a consistent improvement in MI across all models, indicating that the refactored outputs were generally more maintainable than the original code. Both *CodeLlama* and *WizardCoder* demonstrated measurable gains across all prompt settings. Notably, the MI for *StarCoder 2* remained nearly unchanged in the one-shot and few-shot settings, suggesting that while the model could occasionally

produce syntactically valid refactorings, those changes did not meaningfully affect the maintainability of the code.

These results reinforce earlier findings that *CodeLlama* and *WizardCoder* not only produce correct and concise code but also improve its long-term readability and structural quality—both critical for downstream maintenance and human review. [Table 2]

3) Levenshtein Distance (Refactoring Aggressiveness)

To quantify the extent of changes introduced by the models during refactoring, we computed the *Levenshtein Distance* between the original and refactored code for each accepted solution. Higher values indicate more substantial modifications.

Zero-shot prompting consistently produced the most aggressive refactorings across all models, with the highest *Levenshtein Distances* observed in this configuration. For instance, *CodeLlama's zero-shot* refactorings had a noticeably higher character-level edit distance compared to its one-shot and *few-shot* counterparts. Similarly, *WizardCoder* introduced significantly larger structural and syntactic changes under *zero-shot* conditions, whereas its other prompt settings yielded more restrained edits.

One-shot and *few-shot* prompts led to more conservative transformations, suggesting that with additional context, models tend to preserve more of the original structure and make minimal, targeted changes instead of overhauls.

An interesting exception emerged with *StarCoder2*. While the model generally struggled with meaningful transformations, its *zero-shot Levenshtein* values were non-zero, indicating that in a few rare successful cases, it made minor modifications beyond trivial formatting. However, in *one-shot* and *few-shot* settings, *Levenshtein* values dropped close to zero—consistent with earlier findings that the model was only capable of handling extremely simple or near-verbatim refactorings.

These results help distinguish not only which models are effective, but also how they behave when altering source code—highlighting the trade-off between correctness and the degree of modification, which is particularly relevant in production settings where excessive code churn may be undesirable. [Table 2]

4) Developer survey

a) Participant Demographics

To assess developer perception of refactored code quality, we conducted a survey involving 15 participants with varying levels of software development experience and Python proficiency. Participants were categorized by experience level(Junior: 0–3 years, Mid: 3–7 years, Senior: 7+ years) and self-reported Python skill (Beginner, Intermediate, Expert).

The participant pool was evenly split between Mid-level and Senior developers, each group comprising 7 participants, while only 1 participant fell under the Junior category. In terms of Python expertise, the majority identified as Intermediate users (12), with 2 Beginners and only 1 Expert.

This distribution provided a balanced perspective from experienced practitioners while minimizing bias toward either novices or highly specialized Python experts.

b) Overall Preference Trends

Refactored code was selected slightly more often (61 votes) than the original (53 votes), with 36 responses indicating no clear preference. This indicates a mild overall favorability toward refactored outputs, though the relatively small margin and high number of neutral responses suggest that many refactorings did not result in clear improvements from the developer's perspective.

c) Preference Rationale

Readability emerged as the most frequently cited factor for both refactored and original selections. Among responses favoring refactored code, readability was cited in 40 cases, followed by conciseness (30) and maintainability (25). In contrast, selections favoring the original code emphasized readability even more strongly (45 mentions), followed by maintainability (25), with conciseness cited only 9 times.

These results suggest that while refactored code was often recognized for being more concise, it was not always viewed as more readable. When refactorings compromised code clarity or introduced unfamiliar structure, participants were more likely to prefer the original version.

d) Implications

Overall, the findings reinforce that developer preference is not solely influenced by functional correctness or reduced verbosity. Instead, readability plays a dominant role in how refactored code is judged. Refactorings that improve conciseness without sacrificing clarity are generally well received, whereas aggressive transformations that obscure logic tend to be rejected—even if they offer technical improvements. These insights highlight the importance of aligning automated refactoring techniques with human-centered principles, particularly in contexts where code will be reviewed, maintained, or extended by other developers.

C. RQ 3 - Impact of Prompting Strategy

Our evaluation across all three prompting strategies—*zero-shot, one-shot,* and *few-shot*—demonstrates that prompt engineering plays a critical role in the success of LLM-driven code refactoring. *Few-shot* prompting emerged as the most effective configuration overall, especially for *CodeLlama*, where it led to the highest correctness scores (*Pass@5* = 0.70), reduced *cyclomatic complexity*, higher *maintainability index*, and more structured and concise implementations. The added examples in *few-shot* settings appear to provide crucial inductive bias that helps the model generate not only more accurate but also cleaner and more structurally sound refactorings.

One-shot prompting generally served as a middle ground—offering modest improvements in maintainability and structural metrics over zero-shot, while still falling short of the performance achieved with few-shot inputs.

In contrast, *zero-shot* prompting led to the most aggressive and unrestrained code transformations, as evidenced by the highest *Levenshtein distances* across models. While this sometimes resulted in significant *LOC reduction* or simplification, it also correlated with increased cyclomatic complexity and lower correctness overall, particularly in the case of *WizardCoder* and *StarCoder 2*.

Notably, *StarCoder 2* showed minimal sensitivity to prompting strategy, failing to exhibit meaningful variation in correctness or maintainability across all three configurations.

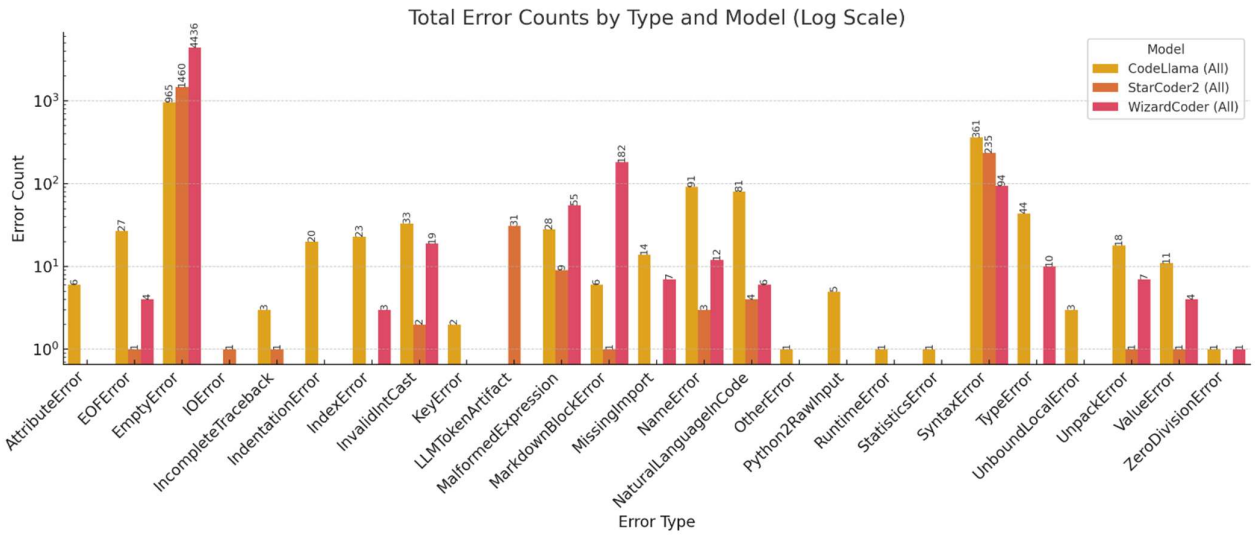

Figure 1: Distribution of Refactoring Errors by Type and Model (Log Scale)

This further underscores its limitations in adapting to code-refactoring tasks through prompt-based conditioning.

These findings highlight that prompting is a key determinant of model success in automated refactoring. Models like *CodeLlama* exhibit a high degree of prompt responsiveness, while weaker models remain largely invariant to the structure or quality of the prompt. Effective prompt engineering, especially using *few-shot* examples, can dramatically enhance refactoring outcomes and should be considered a core component of any LLM-based software maintenance pipeline.

D. Error Analysis

Given the large number of incorrect outputs identified by the virtual judge, we conducted a targeted error analysis to better understand the failure modes of each model. We categorized and counted the types of execution errors encountered across all failed refactored programs, with results shown in log scale in the figure above.

Across all models, the most prevalent error category was *EmptyError*, indicating that the model failed to return any usable output. This issue occurred most frequently with *WizardCoder* (4,436), followed by *StarCoder2* (1,460) and *CodeLlama* (965). This suggests that even when prompted correctly, models may occasionally fail to generate code altogether, particularly under more complex or ambiguous conditions. [Figure 1]

Another prominent error type was *SyntaxError*, with *CodeLlama* and *StarCoder2* recording *3,561* and *2,355* occurrences respectively, while *WizardCoder* had *944*. This reflects common issues with improperly terminated statements, mismatched brackets, or malformed indentation—errors likely stemming from token prediction failures or formatting mismatches in the LLM outputs. Other frequently observed errors included:

- *TypeError* and *ValueError*, indicating problems with incorrect data types or function arguments.
- *MissingImport*, which often resulted from the model removing or overlooking necessary library imports during refactoring.
- *MalformedExpression* and *MarkdownBlockError*, both of which likely originate from LLMs misinterpreting or inserting markdown-style formatting into code (e.g., when ignoring instructions to omit explanations or quotes).
- *NaturalLanguageInCode* errors, especially in *WizardCoder*, which suggest the model occasionally inserted plain language text or commentary directly into the output instead of executable code.

Interestingly, *CodeLlama*, despite having the most correct outputs overall, still produced a large volume of runtime-related issues, such as IndexError, KeyError, and *UnboundLocalError*, often stemming from variable scope mismanagement or aggressive simplifications.

This analysis reinforces the importance of rigorous validation when applying LLMs to automated refactoring. Even strong models like *CodeLlama* produce a non-negligible number of syntactically valid but semantically incorrect programs.

VIII. Discussion

A. Takeaways

Our study reveals several key insights into the current capabilities and limitations of Large Language Models in automated code refactoring.

- *CodeLlama* shows the most balanced performance. Among the evaluated models—*CodeLlama*, *StarCoder2*, and *WizardCoder*—*CodeLlama* consistently demonstrated the most reliable outcomes across all core metrics, including functional correctness, code reduction, and maintainability. Its refactored outputs exhibited a strong trade-off between conciseness and structural integrity, making it the most

dependable model for general-purpose refactoring tasks.

- *Prompt engineering* significantly influences outcomes. Our experiments showed that few-shot prompting led to superior refactoring quality compared to zero-shot approaches. Providing the model with clear, diverse examples helped it understand the intent and pattern of desired transformations, thereby reducing the likelihood of semantic errors and improving maintainability. This highlights the importance of prompt design as a critical factor in practical deployments of LLM-based tooling.
- *Automated refactoring* is promising but not yet production ready. Despite achieving high success rates in functional correctness, LLMs still frequently produce outputs with syntactic issues or flawed logic, especially in edge cases or under minimal prompting. These inconsistencies underline the need for post-processing validation mechanisms or human oversight to ensure safe integration into professional development pipelines.
- *Human preferences* align with quantitative metrics. Feedback from human evaluators indicated a strong preference for outputs that ranked higher in our automated evaluations, particularly in terms of readability, simplicity, and maintainability. This alignment validates the effectiveness of our metric-driven evaluation framework and supports the use of such automated assessments in future studies and tool development.

These takeaways show the growing potential of LLMs to support software refactoring, while also identifying critical areas—such as robustness, prompting strategies, and verification—that must be addressed to fully realize their utility in real-world software engineering.

B. Lessons Learned

In the course of conducting this study, we encountered a number of practical challenges and insights that are important for future research and deployment of LLM-based code tools:

1) Running models locally requires substantial compute power.

Despite using 7B parameter models, inference required long runtimes and GPU memory management. Our hardware setup (*Intel Core i9, RTX 4050*) was sufficient for experimentation, but scaling to larger models or datasets would require more powerful infrastructure or cloud-based solutions.

2) LLM outputs are highly variable and error-prone.

Even top-performing models like CodeLlama frequently produced empty responses, invalid syntax, or misinterpreted markdown artifacts. This reinforces the need for robust error-handling, post-processing scripts, and fallback mechanisms when deploying models at scale.

3) Checkpoints are critical for batch processing.

When running long pipelines across hundreds of programs and prompt variations, implementing checkpoints proved essential. They prevented data loss during interruptions, reduced re-computation, and ensured consistency in model output evaluation.

4) Systematic evaluation pipelines improve reproducibility.

By modularizing our scripts for prompt generation, model querying, output parsing, metric calculation, and correctness evaluation, we were able to streamline experimentation and improve reproducibility across multiple runs.

5) LLMs are not deterministic.

Repeated queries on the same input often resulted in different outputs, even with identical prompts. This makes result tracking and comparison more complex and emphasizes the value of *Pass@k* metrics and storing all model outputs for analysis.

6) Prompt instructions are frequently ignored.

Despite explicitly instructing models to return only raw Python code, some outputs still contained natural language, markdown formatting, or explanations. This shows that instruction-following in code contexts can still be fragile and suggests a need for stricter decoding strategies or cleaner prompt design.

Collectively, these lessons emphasize that while LLM-based refactoring tools show great potential, practical deployment requires careful engineering, infrastructure planning, and robust evaluation practices. These findings will help guide future work in scaling, debugging, and refining LLM applications in software engineering.

IX. Threats to Validity

While our study offers meaningful insights into the use of Large Language Models (LLMs) for automated code refactoring, several limitations and potential threats to validity must be acknowledged.

A. Construct Validity

Our evaluation relies on a combination of automated metrics (*Pass@k, LOC, CC, Halstead Complexity, Maintainability Index, Levenshtein Distance*) and human preference ratings to measure the effectiveness of refactorings. Although these metrics are widely accepted in software engineering, they may not fully capture all dimensions of code quality—such as naming consistency, idiomatic use of language features, or alignment with specific coding conventions. Furthermore, human judgments were collected using side-by-side comparisons without exposing functional behavior, which may have biased responses toward more superficial aspects such as formatting or brevity.

B. Internal Validity

The models were queried using scripted prompts and evaluated under uniform hardware and software conditions, but non-determinism in LLM outputs introduces variability. Identical prompts can produce different outputs across runs due to sampling randomness. Although we mitigated this by computing *Pass@k* and storing multiple generations per sample, internal validity may still be impacted by stochastic behavior and prompt sensitivity. Additionally, some model responses included markdown artifacts or comments despite instructions, potentially influencing success rates and automated metric results.

C. External Validity

Our dataset is composed of 420 Python programs drawn from the Aizu Online Judge platform's introductory programming problems. While this ensures functional correctness and diversity across basic control structures and algorithms, the findings may not generalize to more complex, real-world projects with deeper architectural dependencies or domain-specific constraints. Moreover, the study only evaluates Python-based refactoring, limiting applicability to other languages and environments.

D. Reliability

All scripts and evaluation components were modularized and version-controlled to ensure reproducibility. However, inference performance may vary with different model versions, hardware configurations, or runtime environments. The use of locally hosted 7B models also imposes constraints on model size and capability, and results might differ when using more powerful or fine-tuned variants. Despite this, our setup reflects a practical baseline for academic and resource-constrained use cases.

While our methodology was designed to provide rigorous and comprehensive insights, these threats highlight areas where future work could strengthen the generalizability and robustness of LLM-based refactoring evaluations—particularly through larger and more diverse datasets, cross-language comparisons, fine-tuned models, and deeper integration of developer workflows.

X. Conclusion

This study presents a comparative evaluation of Large Language Models—specifically *CodeLlama, StarCoder2*, and *WizardCoder*—for the task of automated code refactoring. Through a combination of functional correctness checks, static code analysis, and human evaluation, we assessed the effectiveness of each model in improving code quality while preserving original behavior.

CodeLlama emerged as the most effective model, consistently outperforming *StarCoder2* and *WizardCoder* in both correctness and code reduction. Its performance was particularly notable under few-shot prompting conditions, where it produced concise, accurate, and maintainable refactorings across a wide range of inputs.

Prompt engineering proved to be a decisive factor in refactoring success. Across all models, few-shot prompting led to higher *pass@k* scores and more coherent code transformations. This reinforces the idea that thoughtful example selection and instruction design are critical to guiding LLM behavior in code-related tasks.

In terms of maintainability and complexity, improvements were observed but not uniform. Some models produced highly accurate code at the cost of increased structural complexity or reduced readability. These trade-offs highlight the need for balancing functional precision with code simplicity, particularly in real-world software engineering environments.

Human evaluators generally favored refactored outputs that offered meaningful changes without excessive restructuring. Outputs that were cleaner, better organized, and easier to understand—yet still familiar—were rated higher, validating the importance of aligning automated metrics with developer expectations.

Finally, common failure modes such as syntax errors and semantic inconsistencies revealed the current limitations of LLMs in reliably maintaining structural code integrity. These errors, while infrequent, emphasize the need for improved post-processing, testing pipelines, and possibly hybrid systems that combine LLM suggestions with traditional static analysis.

In summary, while LLMs like *CodeLlama* demonstrate strong potential for automated refactoring, practical deployment still requires careful prompt design, post-validation, and a collaborative human-in-the-loop approach to ensure code quality and reliability. Future work may explore larger models, fine-tuning on domain-specific data, and integration into existing development tools to further advance this promising direction.

References

[1] M. Fowler, Refactoring: Improving the Design of Existing Code. Addison-Wesley, 1999.

[2] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," Nature, vol. 323, no. 6088, pp. 533–536, 1986.

[3] A. Ziegler et al., "Efficiency and scalability of large language models in code generation," in Proc. ICSE, 2022, pp. 1–12.

[4] C. Diggs et al., "Leveraging LLMs for Legacy Code Modernization: Challenges and Opportunities for LLM-Generated Documentation," arXiv preprint arXiv:2411.14971, 2024. Available: https://arxiv.org/abs/2411.14971

[5] B. Liu, Y. Jiang, Y. Zhang, N. Niu, G. Li, and H. Liu, "An Empirical Study on the Potential of LLMs in Automated Software Refactoring," arXiv preprint arXiv:2411.04444, 2024. Available: https://arxiv.org/abs/2411.04444

[6] A. Shirafuji, Y. Oda, J. Suzuki, M. Morishita, and Y. Watanobe, "Refactoring Programs Using Large Language Models with Few-Shot Examples," arXiv preprint arXiv:2311.11690, 2023. Available: https://arxiv.org/abs/2311.11690

[7] M. Kim, T. Zimmermann, and N. Nagappan, "An empirical study of refactoring challenges and benefits at Microsoft," IEEE Transactions on Software Engineering, vol. 40, no. 7, pp. 633–649, July 2014. Available: https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/kim-tse-2014.pdf

[8] B. F. Békefi, K. Szabados and A. Kovács, "A case study on the effects and limitations of refactoring," 2019 IEEE 15th International Scientific Conference on Informatics, Poprad, Slovakia, 2019, pp. 000213-000218. doi: 10.1109/Informatics47936.2019.9119321.

[9] A. Tornhill, "Refactoring vs refuctoring: Advancing the state of AI-automated code improvements," CodeScene, 2022. Available: https://codescene.com/hubfs/whitepapers/Refactoring-vs-Refuctoring-Advancing-the-state-of-AI-automated-code-improvements.pdf

[10] Y. Watanobe, "Aizu Online Judge," 2004.

[11] OpenAI, "Gpt-4 technical report," arXiv preprint, 2023.

[12] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, et al., "Llama: Open and efficient foundation language models," arXiv preprint, 2023.

[13] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, et al., "Starcoder: may the source be with you!," arXiv preprint, 2023.

[14] Z. Luo, C. Xu, P. Zhao, Q. Sun, X. Geng, et al., "Wizardcoder: Empowering code large language models with evol-instruct," arXiv preprint, 2023.

[15] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, et al., "Evaluating large language models trained on code," arXiv preprint, 2021.

[16] T. McCabe, "A complexity measure," IEEE Transactions on Software Engineering, vol. SE-2, no. 4, pp. 308–320, 1976.

[17] Halstead, Maurice H. (1977). Elements of Software Science. Amsterdam: Elsevier North-Holland, Inc.

[18] Microsoft, "Code metrics maintainability index range and meaning," Microsoft Learn, 2022. [Online]. Available: https://learn.microsoft.com/en-us/visualstudio/code-quality/code-metrics-maintainability-index-range-and-meaning?view=vs-2022. [Accessed: 2025-03-03].

[19] V. I. Levenshtein, "Binary Codes Capable of Correcting Deletions, Insertions and Reversals," Soviet Physics Doklady, vol. 10, p. 707, 1966.

[20] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, et al., "Language models are few-shot learners," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 1877–1901, 2020.

[21] Wei, Jason; Wang, Xuezhi; Schuurmans, Dale; Bosma, Maarten; Ichter, Brian; Xia, Fei; Chi, Ed H.; Le, Quoc V.; Zhou, Denny (October 31, 2022). "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." Advances in Neural Information Processing Systems (NeurIPS 2022), vol. 35. arXiv:2201.11903.

[22] R. Puri, D. S. Kung, G. Janssen, W. Zhang, G. Domeniconi, et al., "CodeNet: A large-scale AI for code dataset for learning a diversity of coding tasks," in Proceedings of the 35th Conference on Neural Information Processing Systems (NeurIPS) Track on Datasets and Benchmarks (Round 2), 2021.

[23] A. R. Ardila Agudelo, D. P. Escalera, and A. Dhim, "LLMRefactoring: Evaluating Large Language Models for Automated Code Refactoring," GitHub repository, 2025. [Online]. Available: https://github.com/AndresArdila544/LLMRefactoring

[24] Ollama, "Ollama: Run large language models locally," 2023. [Online]. Available: https://ollama.com/

[25] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, et al., "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020. [Online]. Available: https://arxiv.org/abs/2005.14165

[26] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022. [Online]. Available: https://arxiv.org/abs/2201.11903

[27] J. K. White, A. Ouyang, A. Jain, Y. Taori, J. Hernandez, et al., "Prompt Engineering Techniques for Large Language Models: A Survey," arXiv preprint arXiv:2307.06435, 2023. [Online]. Available: https://arxiv.org/abs/2307.06435

[28] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, vol. 10, pp. 707–710, 1966.

Author Contributions (CRediT Taxonomy + Specific Responsibilities)

ANDRES RICARDO ARDILA AGUDELO

CRediT Roles: Conceptualization, Data Curation and Analysis, Experiments, Validation, Writing – Original Draft, Writing – Review & Editing

Responsibilities: Tailored the dataset; set up the virtual judge system; configured the virtual machine for resource-intensive models; ran all experiments; structured the developer survey; contributed to writing the report and presentation.

DIEGO PALACIOS ESCALERA

CRediT Roles: Conceptualization, Experiments, Validation, Writing – Original Draft, Writing – Review & Editing

Responsibilities: Designed the refactoring prompts; ran CodeLlama and StarCoder2 experiments; duplicated and validated experimental results; supported survey structure; contributed to writing and editing the report and presentation.

AHKIL DHIM

CRediT Roles: Conceptualization, Validation, Visualization, Writing – Original Draft, Writing – Review & Editing

Responsibilities: Ran WizardCoder experiments; assisted with survey design; processed and analyzed survey responses; helped with the draft and review of the final report.